

Competitive Exam Practice 1: Foundations, Pointers & ADTs

GATE (Computer Science) Pattern

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 13, 2026

Each question carries 1–2 marks. MCQ = single correct option; NAT = Numerical Answer Type; TF = True/False. C is the language.

Q1 [*GATE CSE 2004 Q1, verified — GATEOverflow V2 p.308*] [**MCQ, 1 mark**] The goal of structured programming is to:

- (A) have well indented programs.
- (B) be able to infer the flow of control from the compiled code.
- (C) be able to infer the flow of control from the program text.
- (D) avoid the use of GOTO statements.

Q2 [*Original, GATE-pattern*] [**MCQ, 1 mark**] Consider the following C function:

```
int largest3(int a, int b, int c) {  
    int m = a;  
    if (b > m) m = b;  
    if (c > m) m = c;  
    return m;  
}
```

Which control patterns appear in this function?

- (A) Sequence and iteration only.
- (B) Sequence and selection only.
- (C) Selection and iteration only.
- (D) Sequence, selection, and iteration.

Q3 [*Original, GATE-pattern*] [**MCQ, 1 mark**] In a running C program, where does a local variable declared inside a function live, and where does memory requested by `malloc` live, respectively?

- (A) heap; stack
- (B) stack; heap
- (C) stack; stack
- (D) heap; heap

Q4 [*Original, GATE-pattern*] [**TF, 1 mark**] State whether the following statement is TRUE or FALSE:

A block of memory obtained by `malloc` inside a function survives the return of that function, whereas a local variable of the same function does not.

Q5 [*GATE CS 2001 Q2.18, verified — GATEOverflow V2 p.293*] [**MCQ, 2 marks**] Consider the following three C functions:

```
/* P1 */
int *g(void) {
    int x = 10;
    return (&x);
}
/* P2 */
int *g(void) {
    int *px;
    *px = 10;
    return px;
}
/* P3 */
int *g(void) {
    int *px;
    px = (int *) malloc(sizeof(int));
    *px = 10;
    return px;
}
```

Which of the above three functions are likely to cause problems with pointers?

- (A) Only P3
- (B) Only P1 and P3
- (C) Only P1 and P2
- (D) P1, P2 and P3

Q6 [*GATE CS 2014 Set 1 Q10, verified — GATEOverflow V2 p.295*] [**MCQ, 1 mark**] Consider the following program in C language:

```
#include <stdio.h>
main() {
    int i;
    int *pi = &i;
    scanf("%d", pi);
    printf("%d\n", i + 5);
}
```

Which one of the following statements is TRUE?

- (A) Compilation fails.

- (B) Execution results in a run-time error.
- (C) On execution, the value printed is more than the address of variable `i`.
- (D) On execution, the value printed is more than the integer value entered.

Q7 [*GATE CS 2017 Set 1 Q13, verified — GATEOverflow V2 p.296*] **[MCQ, 1 mark]** Consider the following C code:

```
#include <stdio.h>
int *assignval(int *x, int val) {
    *x = val;
    return x;
}
void main() {
    int *x = malloc(sizeof(int));
    if (NULL == x) return;
    x = assignval(x, 0);
    if (x) {
        x = (int *)malloc(sizeof(int));
        if (NULL == x) return;
        x = assignval(x, 10);
    }
    printf("%d\n", *x);
    free(x);
}
```

The code suffers from which one of the following problems:

- (A) compiler error as the return of `malloc` is not typecast appropriately.
- (B) compiler error because the comparison should be made as `x == NULL`.
- (C) compiles successfully but execution may result in dangling pointer.
- (D) compiles successfully but execution may result in memory leak.

Q8 [*GATE CS 2025 Set 1 Q24, verified — GATEOverflow V2 p.297*] **[NAT, 1 mark]** What is the output of the given C program?

```
#include <stdio.h>
void foo(int *p, int x) {
    *p = x;
}
int main() {
    int *z;
    int a = 20, b = 25;
    z = &a;
    foo(z, b);
    printf("%d", a);
    return 0;
}
```

The output of the given program is _____. (Answer in integer)

Q9 [Original, GATE-pattern] [TF, 1 mark] State whether the following statement is TRUE or FALSE:

NULL is a pointer value meaning “points to nothing,” and dereferencing a NULL pointer is undefined behaviour — in practice, a crash.

Q10 [Original, GATE-pattern] [MCQ, 1 mark] Which of the following statements about `malloc` is TRUE?

- (A) On success `malloc` returns the number of bytes allocated.
- (B) On success `malloc` returns the address of the requested block; on failure it returns NULL.
- (C) `malloc` always returns NULL once the program has used more than half of its memory.
- (D) A pointer obtained from `malloc` must be dereferenced immediately or the program crashes.

Q11 [Original, GATE-pattern] [MCQ, 1 mark] Consider the C statement `int *p = calloc(4, sizeof(int));`, executed successfully, and compare it with the same-sized request `int *q = malloc(4 * sizeof(int));`. Which of the following is TRUE?

- (A) Both `calloc` and `malloc` initialize the block to zero.
- (B) `calloc` initializes every byte of the block to zero; `malloc` leaves the contents unspecified.
- (C) `calloc` leaves the contents unspecified; `malloc` initializes every byte to zero.
- (D) Neither call initializes the block; both leave the contents unspecified.

Q12 [Original, GATE-pattern] [MCQ, 2 marks] A program executes:

```
int *p = malloc(3 * sizeof(int)); /* p[0]=10, p[1]=20, p[2]=30 */
int *q = realloc(p, 6 * sizeof(int));
```

Suppose `realloc` succeeds and returns a block at a *different* address than `p`. Which of the following is TRUE?

- (A) The values 10, 20, 30 are lost; the new block’s first three slots are unspecified.
- (B) `q[0..2]` equal 10, 20, 30 (copied from `p[0..2]`); `q[3..5]` are unspecified.
- (C) The original block at `p` remains valid and must still be freed separately from `q`.
- (D) `realloc` guarantees `q[3..5]` are initialized to zero, just like `calloc`.

Q13 [Original, GATE-pattern] [NAT, 2 marks] On a 64-bit machine where an `int` occupies 4 bytes and a pointer occupies 8 bytes (alignment: fields are aligned to their own size), how many bytes does the following structure occupy?

```
struct node {
    int data;
    struct node *next;
};
```

The number of bytes occupied is _____.

Q14 [*GATE CSE 2005 Q2, verified — GATEOverflow V2 p.198*] [MCQ, 1 mark] An Abstract Data Type (ADT) is:

- (A) same as an abstract class.
- (B) a data type that cannot be instantiated.
- (C) a data type for which only the operations defined on it can be used, but none else.
- (D) all of the above.

Q15 [*Original, GATE-pattern*] [MCQ, 2 marks] Recall Sedgewick’s definition of a data type as “a set of values and a set of operations on those values,” and the Week 1 **bag** ADT supporting `insert(x)`, `member(x)`, and `size()`. Which statement is TRUE?

- (A) “The values are stored in a dynamic array” is part of the bag’s contract.
- (B) `insert(x)`, `member(x)`, and `size()` name the operations of the contract, which does not say whether the bag is implemented with an array or a linked list.
- (C) An ADT is one particular implementation of a data structure.
- (D) The contract must also specify the C code for every operation.